

5. ShopReturn: Online Seller Returns and Refund Operations

Background

A Dhaka-based F-commerce seller ships 400–500 orders a day (60% cash-on-delivery) through a Facebook-page storefront and couriers. Returns are constant: damaged products, wrong sizes, defective units, delayed deliveries, COD refusals, changed minds. Today returns live in a notebook and the seller's memory. Customers ghost or rage-post because "nobody ever told me what happened to my refund."

The problem today

- Every return is an improvised negotiation; statuses like "received, inspecting, refund processing" exist nowhere the customer can see.
- The seller cannot detect patterns: a SKU with a 38% return rate, a delivery area with repeated failures, a courier that damages boxes.
- Refunds sometimes paid without inspection ,direct loss.

Cost of failure

- Unmanaged 15% return rate ≈ 60k–90k/month in courier, repackaging, and dead stock.
- One viral complaint post can cost a 200-order weekend.
- Decisions (fix the size chart? pause COD in Mirpur? claim against the courier?) are currently made on instinct.

Who is involved

Person	Their world	What frustrates them
Customer	Ordered with COD; wants money or replacement fast	Black hole after the return is requested
Ops staff	Receives, inspects, decides	No process; every case custom
Business manager	Margins, courier negotiations, product mix	Can't see reasons or financial impact of returns

What the product must do

1. Let a customer submit a return request in their own words, with photos, and follow it through a clear set of statuses with an explanation visible at every step.
2. Record **exactly what the customer was told and when** ,the trust artifact.
3. Guide operations through inspection: condition on arrival, restockable or not, courier's fault or seller's.
4. Move money visibly: refund method (bKash/Nagad/bank), amount, reference ,visible to the customer on completion.
5. Require a written reason for any rejection, visible to the customer.
6. Give the business manager the pattern views: return rate by product, by area, by courier, by reason ,with taka impact ,early enough to act (fix the size chart, pause COD in a zone, file a courier claim).

Platform capabilities this case will exercise

A strong solution to this case will naturally need to draw on the full range of platform capabilities available — not just one or two. Concretely:

- Separate sign-in and permissions for customers, ops staff, and the business manager, so customers only ever see their own return.
- A structured return record that carries a clear status set from submission through inspection to refund, with an explanation visible at every step.
- Photo evidence attached to the return, and a written reason required for any rejection.
- A permanent, visible log of exactly what the customer was told and when — not just an internal note.
- Refund records (method, amount, reference) tied to the case and visible to the customer on completion.
- Pattern views for management: return rate by product, area, courier, and reason, with financial impact, built from the same case data ops staff already enter.

Access rules the business expects

- Customers see their own returns and updates only.
- Operational inspection detail and customer contact details are internal.
- Financial totals and analytics are for management; a rejected customer sees their reason, not the internal debate.

Where AI should help

Customers write free text, often Banglish: "order #10-4821 er shoe ta box chire geche, ekta shoe er sole alada hoye geche." The AI should extract the structured reason (damaged / wrong size / COD refusal / changed mind / ...), note likely restockability and whether a courier claim looks warranted, and draft the customer-facing update. Ops staff confirm or correct the categorization before it counts, and every customer message remains human-approved.

AI enhancement challenge

The base case asks the AI to extract a structured reason and draft a customer update for one return. Push it further: have the AI watch the growing set of returns and proactively raise a pattern alert — a SKU or courier crossing a return-rate threshold — with a draft explanation of the likely cause, before the manager has to notice it on a dashboard.

Realistic demo case to script

Customer submits the damaged-shoe return with photo → AI extracts the reason; ops correct one field and accepts → inspection (courier's fault, not restockable) → refund ₳1,450 via bKash with reference → customer sees "Refunded." Complication: the manager's dashboard flags SKU SH-022 at 38% return rate, 24 of 31 returns from Mirpur via one courier, and records the three follow-up decisions (size chart, courier claim, COD pause).

What "great" looks like

The manager can answer: "Which product, which area, and which reason is costing us the most, and what have we decided to do about it?"
